

Umgang mit dem Kursmaterial

- Sämtliches Kursmaterial (Texte, Folien, Abbildungen, Aufgaben, Lösungen usw.) unterliegt dem Urheberrecht des Dozenten und/oder weiterer Rechteinhaber.
- Die Nutzung ist ausschließlich den Studierenden der angegebenen Fachhochschule im Rahmen des angegebenen Kurses und nur zu Ausbildungszwecken gestattet.
- Jede weitergehende Nutzung, insbesondere das Veröffentlichen, Vervielfältigen, Verbreiten, Teilen oder öffentliche Zugänglichmachen des Materials (z.B. auf Webseiten, in Cloud-Speichern, per E-Mail, in Präsentationen oder Publikationen), ist ohne ausdrückliche schriftliche Zustimmung untersagt und kann rechtliche Konsequenzen nach sich ziehen.
- Die Nutzung des Kursmaterials in Verbindung mit Sprachmodellen oder KI-Systemen (z.B. LLMs wie ChatGPT) ist nur dann zulässig, wenn sichergestellt ist, dass die Inhalte nicht zum Training, zur Speicherung oder zu einer sonstigen Weiterverarbeitung durch Dritte verwendet werden (wie z.B. dem LLM-Anbieter) und alle oben genannten Nutzungsbeschränkungen eingehalten werden.



Kapitel_07

Interprozesskommunikation

Betriebssysteme

Prof. Dr. Claus Fühner

Fakultät Informatik

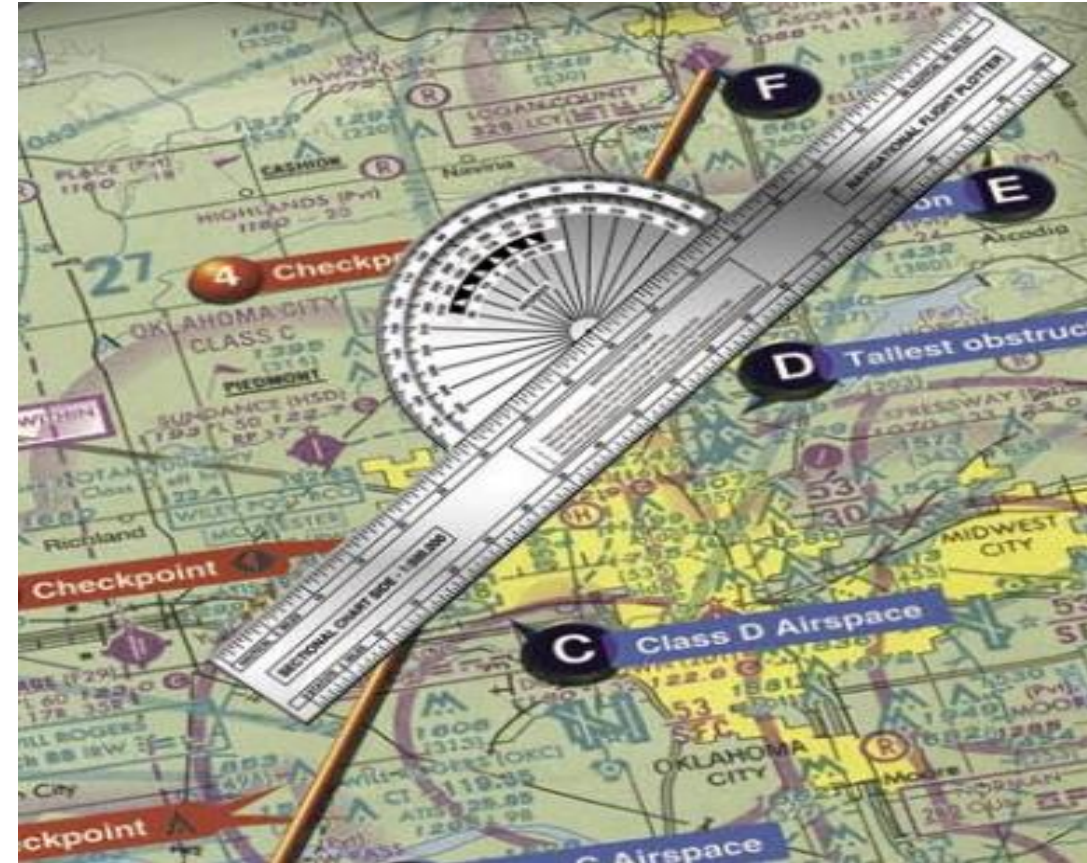
Ostfalia Hochschule für angewandte Wissenschaften

Braunschweig/Wolfenbüttel

Interprozesskommunikation

➔ Nebenläufigkeit und kritische Abschnitte

- Grundlegende IPC-Mechanismen
 - Semaphore und Mutex
 - Condition Variable
 - Monitor
- Weitere IPC-Mechanismen
 - Semaphoren, Shared Memory, Named und Unnamed Pipes/Fifos, Message Passing
- Klassische IPC-Probleme



Ausführungsmodelle

1 CPU

mehr als 1 CPU

sequentiell <-> quasi-parallel <-> parallel

nebenläufig

Nebenläufigkeit (concurrency) bezeichnet parallele oder quasi-parallele Ausführung von Programmen.

- keine echte Parallelität notwendig – präemptives Umschalten auf einem einzigen CPU-Kern reicht aus
- realisiert durch mehrere Prozesse und/oder Threads
- Betriebssystem entscheidet, was wann läuft!
- Schon bei zwei Threads auf einem Prozessor können wir keine Aussage darüber treffen, wie die Anweisungen der beiden Threads ineinander verzahnt werden! Dies ist auch nicht reproduzierbar!
- Wir müssen mit jeder möglichen Verzahnung der Ausführungsstränge rechnen!

Beispiel: Zähler inkrementieren

- Anweisungen aus einem Programmcode:

```
1 c=counter.read()  
2 c++  
3 counter.write(c)
```

- Ablauf in zwei Threads
- Beide Threads greifen auf gleiche Variable counter zu

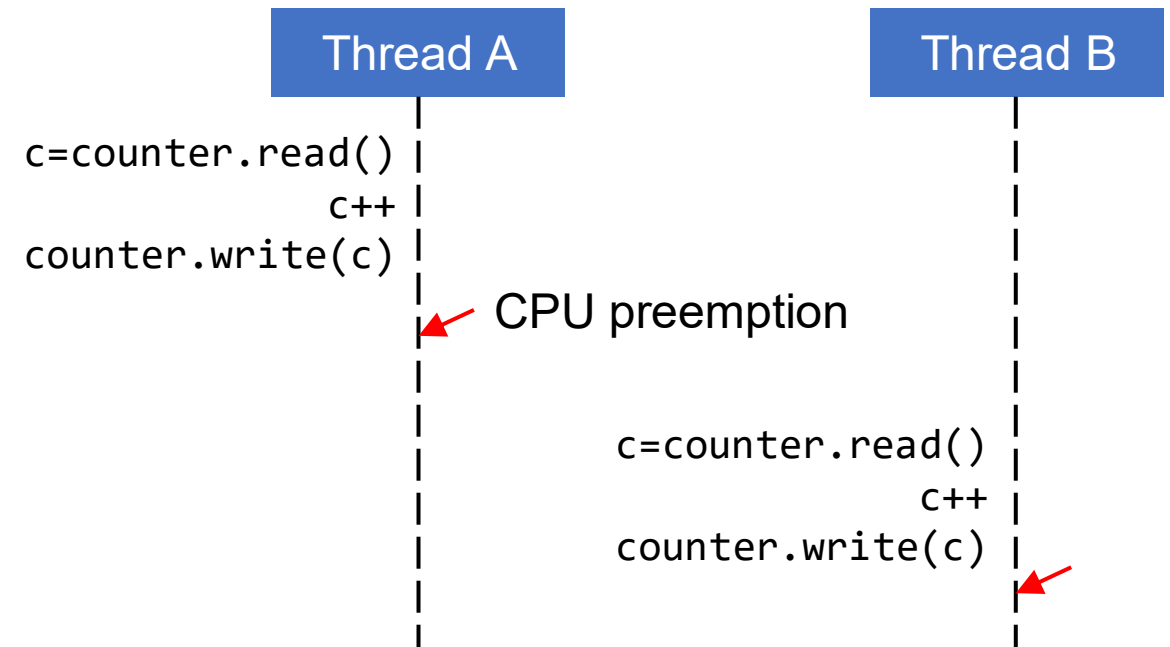


Beispiel: Zähler inkrementieren

- Anweisungen aus einem Programmcode:
- Ablauf in zwei Threads
- Beide Threads greifen auf gleiche Variable counter zu
- Variante 1: Abschnitt in Thread A vollständig durchlaufen, dann Umschaltung auf B
→ counter=2

```
1 c=counter.read()  
2 c++  
3 counter.write(c)
```

Ablaufvariante 1



ähnlich Mandl: Grundkurs Betriebssysteme.

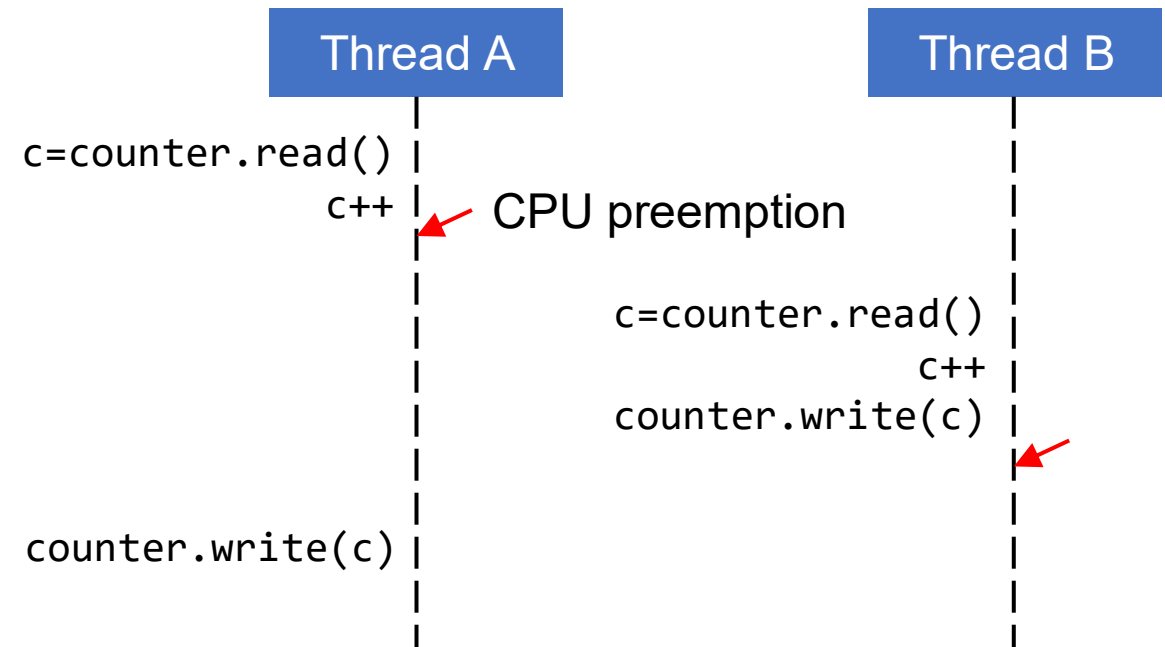


Beispiel: Zähler inkrementieren

- Anweisungen aus einem Programmcode:
- Ablauf in zwei Threads
- Beide Threads greifen auf gleiche Variable counter zu
- Variante 1: Abschnitt in Thread A vollständig durchlaufen, dann Umschaltung auf B
→ counter=2
- Variante 2: Abschnitt in Thread A wird nach Anweisung ② unterbrochen
→ counter=1, **Lost Update**
- Mehrfacher Start des Programms führt zu verschiedenen Ergebnissen!

```
1 c=counter.read()  
2 c++  
3 counter.write(c)
```

Ablaufvariante ②



ähnlich Mandl: Grundkurs Betriebssysteme.

Welche Anweisungen sind atomar?

- Atomare Operation:** Unteilbare Anweisung (oder unteilbare Folge von Einzelanweisungen),
- die entweder ganz oder gar nicht ausgeführt werden
 - die nicht unterbrochen werden können
 - Präemptive Umschaltung in andere Prozesse/Threads nicht möglich
→ Andere Prozesse/Threads können nie auf eine nur teilweise ausgeführte atomare Operation stoßen!
 - Gilt auch für Interrupt Service Routinen (ISRs)!

Was ist atomar?

- **Assembler-Instruktionen** meist atomar, können also nicht unterbrochen werden
- Die meisten aus Operationen aus Hochsprachen werden zu mehreren Assembler-Anweisungen kompiliert und sind deshalb nicht atomar!

Hochsprache (C)
`i++;`

Assembler
`LOAD R1, i`
`ADD R1, 1`
`STORE i, R1`

Nichtatomare Operationen

- Jederzeit mit Unterbrechungen durch andere Tasks/Prozesse/ISRs rechnen!
 - Programmierer kann genaues Scheduling nicht beeinflussen
 - Potentiell ist mit **jeder** Art der Verzahnung der Ausführungsstränge zu rechnen!

Wie kann man eine Folge von Anweisungen atomar machen?

Erster (schlechter) Lösungsansatz:

- Bei Arbeit mit counter die Prozessumschaltung unterbinden
 - Dispatching im Betriebssystem vor ① abschalten ...
 - ... und nach ③ wieder reaktivieren
 - zusätzlich Interrupts sperren, damit auch ISRs während Abschaltung nicht mit counter arbeiten können
- Bewertung
 - Globales Sperren von Interrupts in Multicore- und Multiprozessorsystemen schwierig
 - Multicore-CPU's 1 und 2 könnten die Threads echt parallel ausführen
 - Instabil: Was passiert, wenn ein Prozess mit gesperrten Interrupts in eine Endlosschleife läuft?
 - ➔ Interrupts dürfen meist nur vom Kernel gesperrt werden, sonst Gesamtsystem zu instabil
 - Ineffizient: System stoppt komplett und wartet auf einen einzigen Prozess/Thread

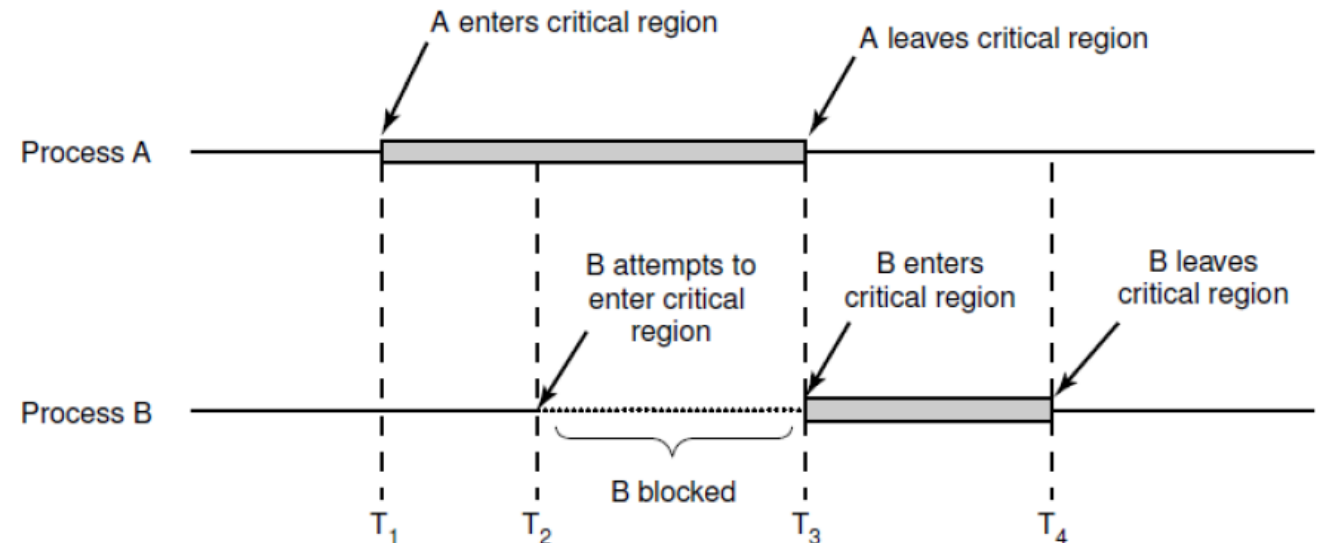
```
① c=counter.read()  
② c++  
③ counter.write(c)
```



Wie kann man eine Folge von Anweisungen atomar machen?

- Eigentlich müssen wir nur verhindern, dass mehrere Threads/Prozesse gleichzeitig mit counter arbeiten
- Der Abschnitt ① - ③ heißt **kritischer Abschnitt bezüglich der Ressource counter**
- Gegenseitiger Ausschluss (Mutual Exclusion):**
Es darf sich immer nur **höchstens ein Thread/Prozess** in einem kritischen Abschnitt bzgl. counter befinden
- Andere Programmteile sind unproblematisch
- Zweiter Thread/Prozess muss warten.
bis erster Thread/Prozess den kritischen Abschnitt wieder freigegeben hat
- Simuliert atomare Anweisungsfolge
(ist aber nicht wirklich atomar!!! - Warum?)
- Aber: Kritische Abschnitt kurz halten, da Flaschenhals (keine Parallelität)

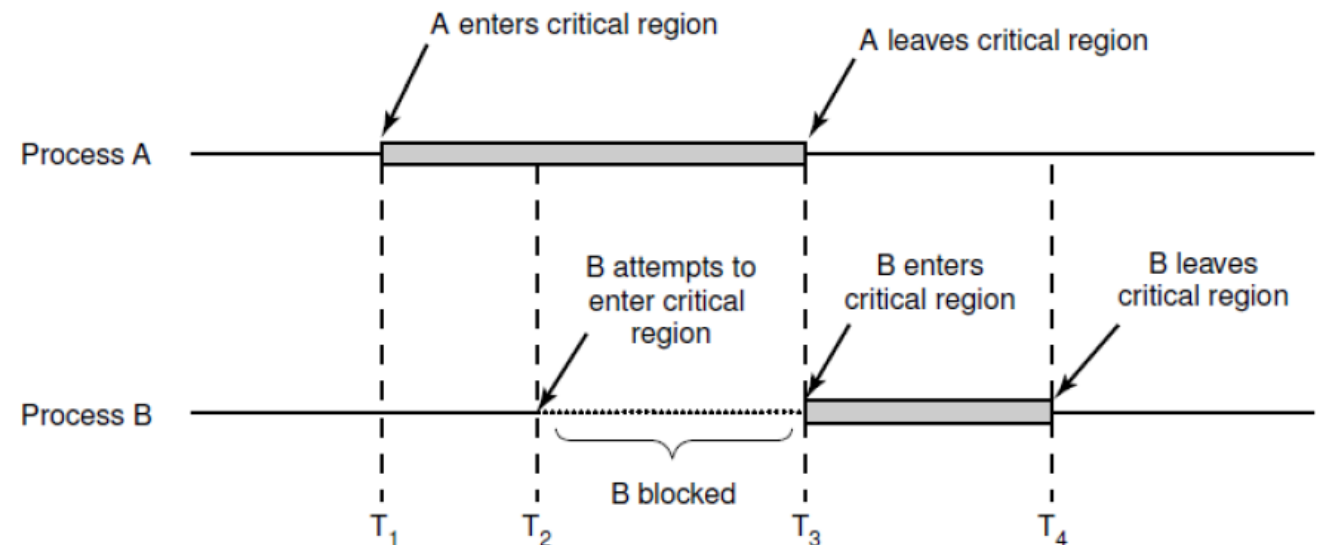
```
1 c=counter.read()  
2 c++  
3 counter.write(c)
```





Grundsätze für kritische Abschnitte (nach Dijkstra)

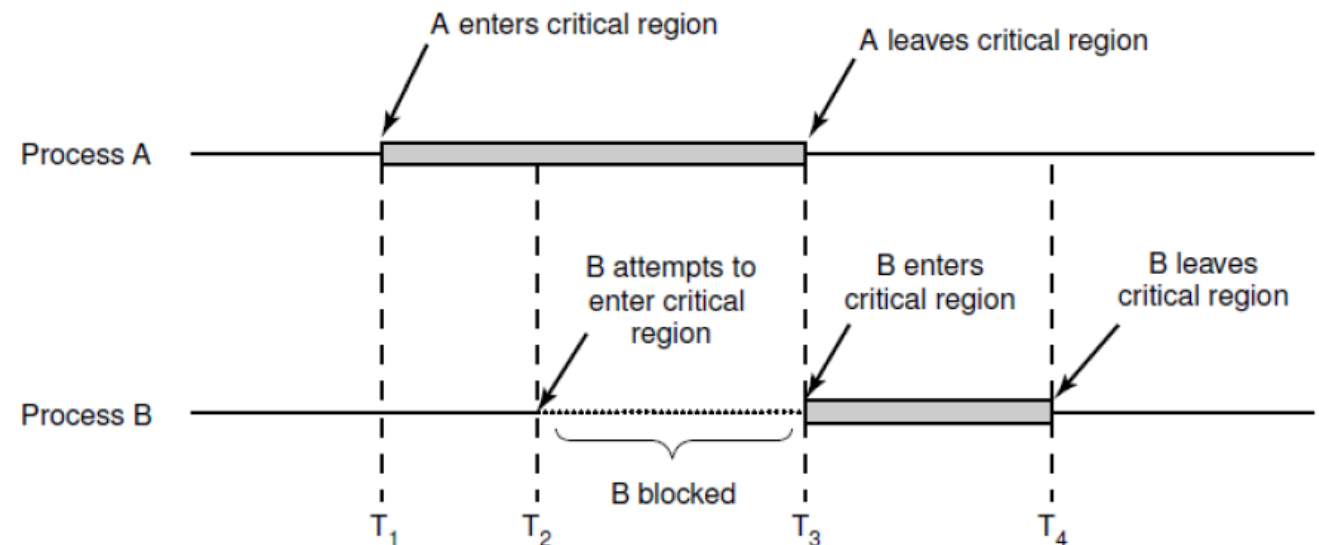
1. Mutual exclusion: Zwei oder mehr Prozesse dürfen sich nicht gleichzeitig im gleichen kritischen Abschnitt befinden.
2. Es dürfen keine Annahmen über die Abarbeitungsgeschwindigkeit und die Anzahl der Prozesse bzw. CPUs gemacht werden. Der kritische Abschnitt muss unabhängig davon geschützt werden.
3. Kein Prozess außerhalb eines kritischen Abschnitts darf einen anderen nebenläufigen Prozess blockieren.
4. Fairness Condition: Jeder Prozess, der am Eingang eines kritischen Abschnitts wartet, muss ihn irgendwann betreten dürfen.
(kein ewiges Warten/Verhungern/Starvation)





Realisierung kritischer Abschnitte

- Realisierung mit Sperrvariable (**Lock**):
 - Sperrvariable = 0: Kein Prozess im kritischen Abschnitt
 - Sperrvariable = 1: Prozess im kritischen Abschnitt, für andere Prozesse gesperrt
- Was tun, wenn kritischer Abschnitt gesperrt?
 - **Aktives Warten (Busy Waiting)** mit **Spinlock**
 - In Schleife ständig abfragen, ob kritischer Abschnitt frei
 - verbrennt CPU-Zeit für Warten
 - Besser: **Prozesszustand Blockiert** zum Warten nutzen
 - Vorteil: andere Threads/Prozesse können die Wartezeit nutzen!



Realisierung kritischer Abschnitte

- Abfragen **und** Setzen der Sperrvariable muss atomar sein
- Softwarelösung möglich (Strikter Wechsel, Decker, Peterson → Tanenbaum: Moderne Betriebssysteme)
- Heute meist spezielle Assembler-Instruktionen wie **Test and Set Lock (TSL)**
 - TSL ermöglicht, **atomar**
 1. den Inhalt der Sperrvariablen in ein Register zu lesen und
 2. einen Wert ungleich Null in die Sperrvariable zu schreiben
 - sperrt in Multicore/-prozessorsystemen auch den Speicherbus, um Zugriff auf Sperrvariable mit anderen CPUs zu synchronisieren
- Implementierung der Sperre mit Spinlock (lässt sich auch in Blocking-Implementierung umschreiben):

Lock:

```
TSL R1, LOCK
CMP R1, #0
JNE Lock
RET
```

Unlock:

```
MOVE LOCK, #0
RET
```

- Was passiert wenn Lock aufgerufen wird bei (1) keiner Sperre und (2) gesetzter Sperre?

Wann kritischer Abschnitt?

- Weiteres Beispiel: Ticketverkauf mit 50 Verkäufern, die concurrent in separaten Threads arbeiten
- Die Funktion sellTicket arbeitet mit einer globalen Variablen remainingTickets:

```
int remainingTickets = 1000;

void sellTicket() {
    while (remainingTickets > 0) {
        sleep_for(500); // simulate "selling a ticket"
        remainingTickets--;
        ...
    }
    ...
}
```

- Wann benötigt man kritische Abschnitte?
 - Bei gemeinsam genutzte Variablen (Ressourcen) ...
 - ... wenn mehrere Threads in die gleiche Variable schreiben (Eingangsbeispiel, lost-update)
 - ... wenn ein Thread in eine Variable schreibt und ≥ 1 Thread die Variable liest (hier)

Hauptprobleme Nebenläufigkeit

- Nebenläufigkeit zu beherrschen ist schwer – insbesondere bei Nutzung gemeinsamer Ressourcen (z.B. Variable)!
- Fehler treten in Tests nur manchmal auf – abhängig davon, wie das Betriebssystem die Tasks gerade abarbeitet
 - ➔ sehr schwer zu reproduzieren und zu debuggen
- Drei Fehlerklassen:
 1. **Race Condition:** Situation, in denen zwei oder mehr Tasks eine gemeinsame Ressource nutzen und das Endergebnis von der (generell unbekannten) zeitlichen Abfolge der Operationen abhängt
 2. **Deadlock (Verklammerung):** Zwei oder mehr Tasks warten wechselseitig aufeinander
 - Gegenseitige Blockade, weil beide Tasks Ressourcen besitzen, die von der jeweils anderen Task benötigt werden
 3. **Starvation (Verhungern):** Rechenbereite Task wird vom Scheduler konsequent übergangen bei unfairer Synchronisationsmechanismus

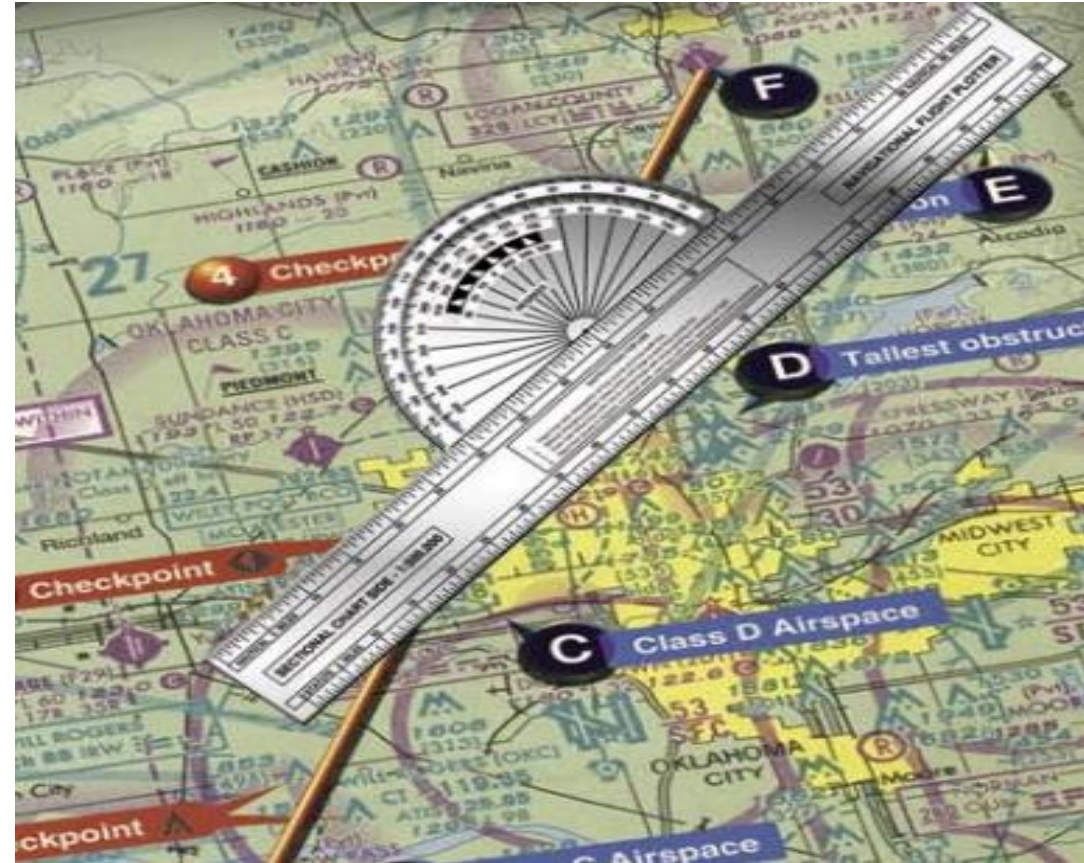
Interprozesskommunikation

Drei Kernprobleme der Interprozesskommunikation (nach Tanenbaum):

1. Wie kann ein Prozess/Thread Information an einen andere Prozess/Thread übermitteln (**Kommunikation**)?
 - Threads nutzen gemeinsamen Adressraum (Eingangsbeispiel)
 - viele weitere Mechanismen (auch für Prozesse)
2. Wie wird sichergestellt, dass sich zwei Prozesse bei kritischen Aktivitäten **nicht gegenseitig in die Quere kommen**?
 - siehe Eingangsbeispiel, Lösung durch Kritische Abschnitte / Mutual Exclusion
3. Wie wird die korrekte Ausführungsreihenfolge sichergestellt, wenn Abhängigkeiten bestehen?
 - Beispiel: Pipe ls | sort - sort-Prozess muss hier auf die Ergebnisse des ls-Prozesses warten

Interprozesskommunikation

- Nebenläufigkeit und kritische Abschnitte
- Grundlegende IPC-Mechanismen
 - Semaphore und Mutex
 - Condition Variable
 - Monitor
- Weitere IPC-Mechanismen
 - Semaphoren, Shared Memory, Named und Unnamed Pipes/Fifos, Message Passing
- Klassische IPC-Probleme



Nur zu Lehrzwecken

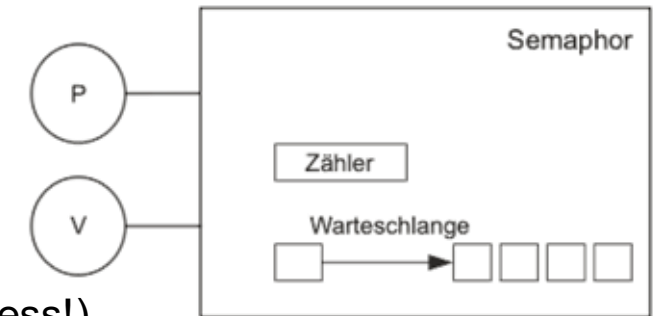


Betriebssysteme



Dijkstra und Semaphoren

- 1965 vom niederländischen Informatiker Edsger W. Dijkstra (1930-2002) publiziert
- Aufgabe: Absicherung von Ressourcen und Synchronisation von Prozessen
- Spezielle, durch das Betriebssystem realisierte **Zählvariable**
- Unterstützt zwei **atomare Operationen P() und V()**
 - **P() / down() / acquire():** probieren, passering
Wenn Zähler > 0: Zähler dekrementieren
Sonst: Prozess **blockiert**, in **Warteschlange** einreihen
 - **V() / up() / release():** vrijgeven
Zähler inkrementieren und ggf. Prozess aus Warteschlange wecken → bereit (Fairness!)
- Typen:
 - **Binäre Semaphore (binary semaphore, mutex):**
Nur die Zählwerte 0 und 1 erlaubt – ideal für kritische Abschnitte!
 - **Zählende Semaphore (counting semaphore, general semaphore):**
Nichtnegative Zählwerte erlaubt – ermöglicht auch komplexere Use Cases
- Weitere subtile Unterschiede, Begriffe Semaphore und Mutex nicht immer sauber abgegrenzt und einheitlich verwendet



Kritische Abschnitte mit Semaphoren

- Kritische Abschnitte lassen sich mit Semaphoren sperren

```
semaphore s_a = 1; // for critical sections, initialize with 1

s_a.p(); // enter critical section
...      // critical operations, e.g. modify complicated data structure
s_a.v(); // leave critical section
```

- Wenn Prozess 1 im kritischen Abschnitt und Prozess 2 einen solchen betreten möchte, blockiert Prozess 2, bis 1 den Abschnitt verlassen hat
- Binäre Semaphoren/Mutexe für Schutz kritische Abschnitte ausreichend
- Andere Probleme (z.B. Producer/Consumer) erfordern zählende Semaphoren

Kritische Abschnitte mit Semaphoren - Pleiten, Pech und Pannen

Verwendung von Semaphoren ist fehlerträchtig:

```
semaphore s = 1;

// do something with res and return true on success
bool doSomethingWithResource()
{
    s.p();           // get exclusive access to resource

    // do something with resource
    // ... and somewhere within this code:
    if (error) {
        return false;
    }

    s.v();           // release exclusive resource access
    return true;
}
```

- Wo steckt hier das Problem? Wie kann man dieses vermeiden?

Kritische Abschnitte mit Semaphoren - Exkurs C++/Rust und RAII

- Pattern in C++ oder Rust: [Ressource Acquisition Is Initialization](#)
- Bindet Lebenszyklus (z.B. P() und V()) an Lebensdauer eines Objekts

```
std::mutex m; // auto-initialized with 1
// std::counting_semaphore m = 1; // alternative: C++ also has semaphores!

void bad()
{
    m.lock();                // acquire the mutex
    f();                      // m never released if f() throws exception
    if (!everything_ok()) return; // early return, mutex never released
    m.unlock();              // release mutex (if reached)
}

void good()
{
    std::lock_guard<std::mutex> lg(m); // RAII wrapper class for mutex, auto-acquires mutex on construction
    f();                             // m always auto-released when leaving scope, ...
    if (!everything_ok()) return;     // ... even on early return or on exception
}                                     // mutex is always released in lock_guard destructor
```

- Unterschied C++/Rust zu Java: Sofortiger Destruktor-Aufruf bei Ende der Lebensdauer (finalize())
- Exceptions in C++ häufig nicht genutzt / verboten und in Rust so nicht vorhanden nach <https://en.cppreference.com/w/cpp/language/raii>

Mehrere verschiedene kritische Abschnitte

- Im gleichen Programm gibt es häufig mehrere kritische Abschnitte bezüglich verschiedener Ressourcen

```
int counter1;  
semaphore sema_counter1 = 1;  
  
void func1() {  
    sema_counter1.P();  
    counter1++;  
    sema_counter1.V();  
}
```

```
int counter2;  
semaphore sema_counter2 = 1;  
  
void func2() {  
    sema_counter2.P();  
    counter2++;  
    sema_counter2.V();  
}
```

- func1 und func2 können in verschiedenen Threads gefahrlos concurrent genutzt werden!
- Aber wenn gleiche Ressource an anderer Stelle genutzt, muss gleiche Semaphorenvariable verwendet werden! Beispiel: `counter1` immer mit `sema_counter1` sichern
- Tipp: Je eine separate Semaphore/Mutex für jede Ressource (Variable) verwenden!
- Vorteil: So erhöhen wir die Parallelität!
- Kann in Java zum Flaschenhals werden, wenn viele Threads die `synchronized` Methoden der gleichen Instanz einer Klasse aufrufen (Java verwendet immer eine Sperrvariable pro Klasse)!

Kritische Abschnitte mit Semaphoren - Deadlock

Zwei
Semaphoren für
zwei
unabhängige
kritische
Abschnitte

```
semaphore resource_1 = 1;  
semaphore resource_2 = 1;  
  
void thread_a() {  
    resource_1.P();  
    resource_2.P();  
    use_both_resources_a();  
    resource_2.V();  
    resource_1.V();  
}  
  
void thread_b() {  
    resource_1.P();  
    resource_2.P();  
    use_both_resources_b();  
    resource_2.V();  
    resource_1.V();  
}
```

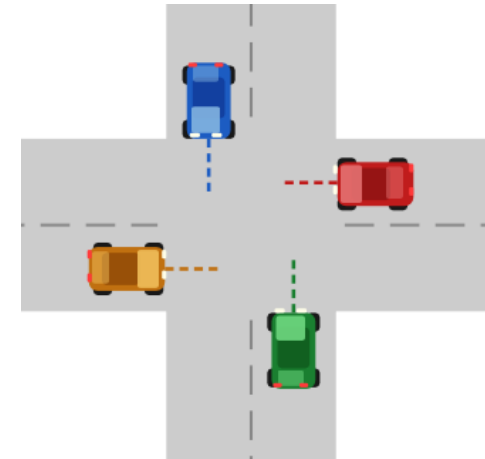
```
semaphore resource_1 = 1;  
semaphore resource_2 = 1;  
  
void thread_a() {  
    resource_1.P();  
    resource_2.P();  
    use_both_resources_a();  
    resource_2.V();  
    resource_1.V();  
}  
  
void thread_b() {  
    resource_2.P();  
    resource_1.P();  
    use_both_resources_b();  
    resource_1.V();  
    resource_2.V();  
}
```

- Deadlock: zwei Threads warten wechselseitig aufeinander
- Abhilfe: Belegungsreihenfolge vorgeben – immer zuerst resource_1 und danach resource_2 belegen
- Bei späterem Hinzubuchen von Ressourcen schwer durchzuhalten!

Kritische Abschnitte mit Semaphoren - Deadlock

Voraussetzungen für Deadlocks (**Coffman-Bedingungen**)

1. **Wechselseitiger Ausschluss:** Ressourcen werden nicht teilbar genutzt, nur ein Prozess kann sie zu einem Zeitpunkt besitzen. Andere Prozesse müssen warten.
 2. **Hold-and-wait-Bedingung:** Prozesse, die schon Ressourcen besitzen, fordern noch weitere Ressourcen an.
 3. **Ununterbrechbarkeit (no Preemption):** Ressourcen, die einem Prozess bewilligt wurden, können diesem nicht gewaltsam wieder entzogen werden. Der Prozess muss sie explizit wieder freigeben.
 4. **Zyklische Wartebedingungen:** Es muss eine zyklische Kette von Prozessen geben, von denen jeder auf eine Ressource wartet, die dem nächsten Prozess in der Kette gehört.
- Alle Bedingungen müssen gleichzeitig erfüllt sein, damit ein Deadlock entstehen kann.
 - Wenn eine Bedingung nicht erfüllt ist, kann es nicht zu einem Deadlock kommen



Rechts vor links!

Übungsbeispiel

- Betrachten Sie die linksstehenden Programme
- Startwerte der Semaphoren: $S_A=1$, $S_B=0$
- Aufgabe a) Statische Prioritäten mit $\text{Prio}(P1) > \text{Prio}(P2)$
- Aufgabe b) Statische Prioritäten mit $\text{Prio}(P2) > \text{Prio}(P1)$
- Aufgabe c) Round Robin Scheduling (Wechsel nach jeder Instruktion)

P1	P2
1 P(S_A)	1 P(S_A)
2 V(S_B)	2 P(S_B)
3 V(S_A)	3 END
4 END	

S_A	S_B	P1	P2

S_A	S_B	P1	P2

S_A	S_B	P1	P2

Klassiker: Producer/Consumer-Problem

- Zwei nebenläufige Prozesse (Producer und Consumer) nutzen zur Kommunikation einen gemeinsamen Puffer fester Größe
- Producer generiert Objekte und legt diese in einen Puffer
- Consumer entnimmt Objekte aus dem Puffer und verarbeitet diese
- Puffer hat eine begrenzte Größe:
 - Producer muss warten, wenn Puffer voll
 - Consumer muss warten, wenn Puffer leer
- Nebstehend/nächste Folie: Falsche Lösung ohne Synchronisation
- Wir
 - starten bei 0 Items
 - produzieren 10 Items (thread P1)
 - konsumieren 10 Items (thread C1)
 - ... und behalten als Resultat 3 Item über ???

Producer/Consumer ohne Sync

(Kapazität: 5)

P1 0: 0 -> 1

C1 0: 1 -> 0

P1 1: 0 -> 1

C1 1: 1 -> 0

P1 2: 1 -> 2

C1 2: 2 -> 1

P1 3: 2 -> 3

C1 3: 1 -> 0

P1 4: 3 -> 4

C1 4: 4 -> 3

P1 5: 4 -> 5

C1 5: 3 -> 2

P1 6: 2 -> 3

C1 6: 2 -> 1

P1 7: 3 -> 4

C1 7: 1 -> 0

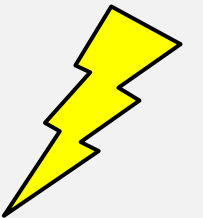
P1 8: 4 -> 5

C1 8: 5 -> 4

P1 9: 4 -> 5

C1 9: 4 -> 3

Final num_items = 3
(sollte 0 sein)



Klassiker: Producer/Consumer-Problem (**falsche** Lösung ohne Synchronisation)

```
void producer(int id, int iterations) {  
    for (int i = 0; i < iterations; ++i) {  
        // produce item (not really do anything)  
        // wait for space in the buffer, then add item  
        while (num_items >= capacity) {  
            // yield for consumer while waiting  
            std::this_thread::yield();  
        }  
        int before = num_items;  
        std::this_thread::yield();  
        num_items = before + 1;  
        std::printf("P%d %3d: %d -> %d\n",  
            id, i, before, num_items);  
    }  
}
```

```
void consumer(int id, int iterations) {  
    for (int i = 0; i < iterations; ++i) {  
        // wait item, then remove it  
        while (num_items <= 0) {  
            // yield for producer while waiting  
            std::this_thread::yield();  
        }  
        int before = num_items;  
        std::this_thread::yield();  
        num_items = before - 1;  
        // consume item (not really do anything)  
        std::printf("C%d %3d: %d -> %d\n",  
            id, i, before, num_items);  
    }  
}
```

- read-modify-write für `num_items++` auseinandergezogen, um den Fehler wahrscheinlicher zu machen
- `yield()` eingefügt, um Taskwechsel (und damit concurrency-Fehler) wahrscheinlicher zu machen
- Fehler treten aber auch ohne `yield()` auf – nur seltener!
- Kritischen Abschnitt um `num_items=...; printf(...);` ergänzen, damit Ausgabe nicht durcheinanderkommt

Klassiker: Producer/Consumer-Problem (Lösung mit drei Semaphoren)

- Full-Semaphore zählt belegte Plätze und blockiert Consumer, wenn keine Items vorhanden (Puffer leer)
- Empty-Semaphore auf Anzahl freier Plätze initialisieren und blockiert Producer, wenn Null (Puffer voll)
- Mutex-Semaphore schützt read-modify-write für num_items

```
// Globale Initialisierung
```

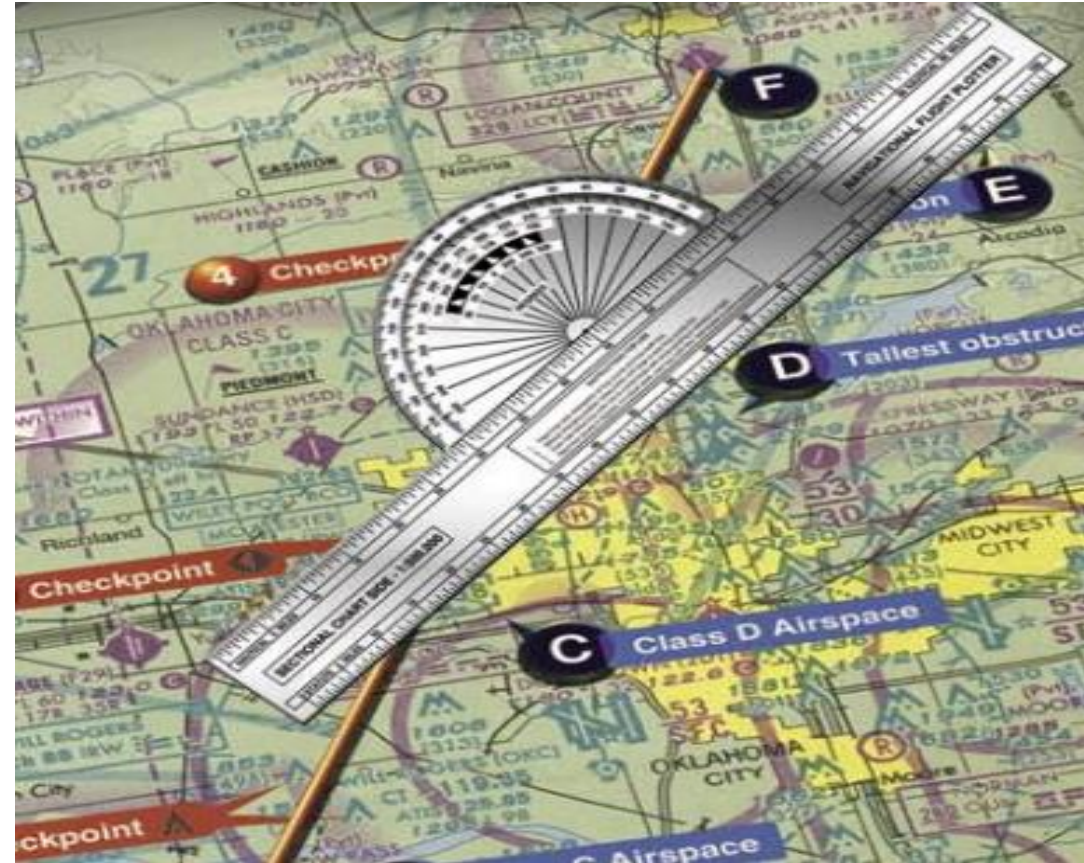
```
std::counting_semaphore full_slots(0);  
std::counting_semaphore empty_slots(capacity);  
std::counting_semaphore mutex(1);
```

```
void producer(int id, int iterations) {  
    for (int i = 0; i < iterations; ++i) {  
        // produce item (not really do anything)  
        empty_slots.acquire(); // block until space  
        std::this_thread::yield();  
        mutex.acquire();  
        int before = num_items;  
        num_items = before + 1;  
        std::printf("P%d %3d: %d -> %d\n",  
            id, i, before, num_items);  
        mutex.release();  
        full_slots.release();  
    }  
}
```

```
void consumer(int id, int iterations) {  
    for (int i = 0; i < iterations; ++i) {  
        full_slots.acquire(); // block until item  
        std::this_thread::yield();  
        mutex.acquire();  
        int before = num_items;  
        num_items = before - 1;  
        // consume item (not really do anything)  
        std::printf("C%d %3d: %d -> %d\n",  
            id, i, before, num_items);  
        mutex.release();  
        empty_slots.release();  
    }  
}
```

Interprozesskommunikation

- Nebenläufigkeit und kritische Abschnitte
- Grundlegende IPC-Mechanismen
 - Semaphore und Mutex
 - Condition Variable
 - Monitor
- Weitere IPC-Mechanismen
 - Semaphoren, Shared Memory, Named und Unnamed Pipes/Fifos, Message Passing
- Klassische IPC-Probleme



Warten auf Bedingungen ohne Busy-Wait

Problem: Thread muss auf eine Bedingung warten

- Beispiel für Bedingung: Puffer nicht leer
- Busy-Wait (Polling der Bedingung in Schleife) verschwendet CPU-Zeit
- Semaphoren passen nur, wenn die Bedingung sich auf das Zählen beschränkt

Idee: Condition Variable

- Thread legt sich schlafen, ...
- ... bis er von einem anderen Thread benachrichtigt (und damit geweckt) wird
- Condition Variable immer an einen Mutex gekoppelt, der den geprüften Zustand schützt
- Erlaubt beliebige *Prädikate* als Wartebedingung (nicht nur Zähler)
 - *Prädikat*: Funktion, die Eingabewerte auf einen Wahrheitswert abbildet

Condition Variable mit C++

Operationen

- `while (!predicate) { wait(lock); }`
Prädikat prüfen und atomar Mutex freigeben, blockieren bis geweckt, Mutex belegen; erneut prüfen ...
 - `wait(lock, predicate)` – Kurzform für die obige Schleife
- `notify_one()` – weckt einen (beliebigen) wartenden Thread
- `notify_all()` – weckt alle wartenden Threads

Standard-Idiom (Warten auf Prädikat)

- Mutex für geteilten Zustand mit `std::unique_lock<std::mutex> my_lock(mutex)` sperren
- In `while`-Schleife Bedingung (basierend auf geteiltem Zustand) prüfen; falls falsch: `wait()` aufrufen
 - *Schutz vor Spurious Wakeups und gegen zwischenzeitliche Zustandsänderungen*
- Zustandsänderung: Mutex halten, dann `notify_one()/notify_all()` aufrufen und Mutex freigeben

Merke: Condition Variable selbst zustandslos – der Zustand kommt aus den Variablen, die der Mutex schützt

Producer/Consumer-Problem mit Condition Variable

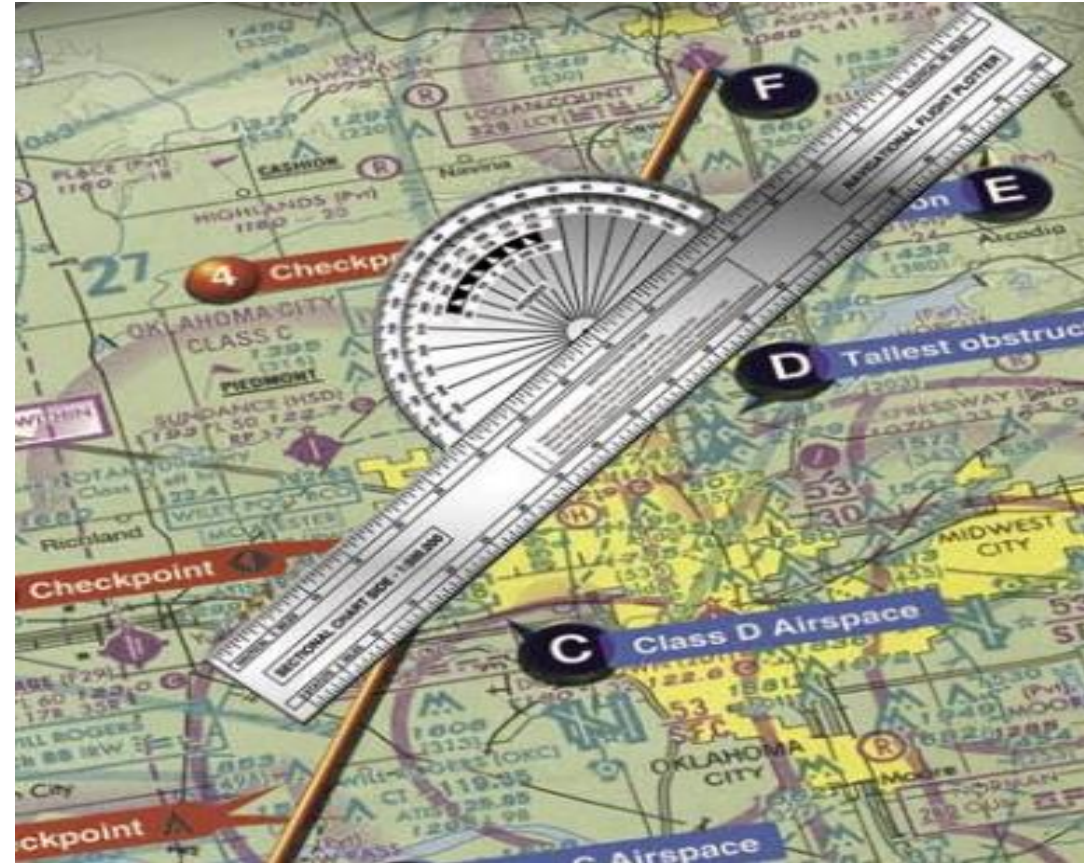
```
std::mutex m;  
std::condition_variable not_full;  
  
void producer(int id) {  
    while (true) {  
        std::unique_lock<std::mutex> lock(m);  
        while (queue.size() == capacity)  
            not_full.wait(lock); // wait with m unlocked  
        queue.push(make_item(id));  
        lock.unlock();  
        not_empty.notify_one();  
    }  
}
```

```
std::condition_variable not_empty;  
  
void consumer(int id) {  
    while (true) {  
        std::unique_lock<std::mutex> lock(m);  
        while (queue.empty())  
            not_empty.wait(lock); //wait with m unlocked  
        auto item = queue.front(); queue.pop();  
        lock.unlock();  
        not_full.notify_one();  
    }  
}
```

- `unique_lock` belegt den Mutex und gibt ihn im Destruktor wieder frei (ähnlich `lock_guard`)
- `wait(lock)` **gibt den Mutex frei**, **blockiert den Thread** und **sperrt den Mutex** beim Aufwachen wieder
- Prüfung immer in `while` (nie mit `if`): Spurious Wakeups möglich, Zustand kann sich zwischen `notify` und erneutem Sperren ändern.

Interprozesskommunikation

- Nebenläufigkeit und kritische Abschnitte
- Grundlegende IPC-Mechanismen
 - Semaphore und Mutex
 - Condition Variable
 - Monitor
- Weitere IPC-Mechanismen
 - Semaphoren, Shared Memory, Named und Unnamed Pipes/Fifos, Message Passing
- Klassische IPC-Probleme



Nur zu Lehrzwecken



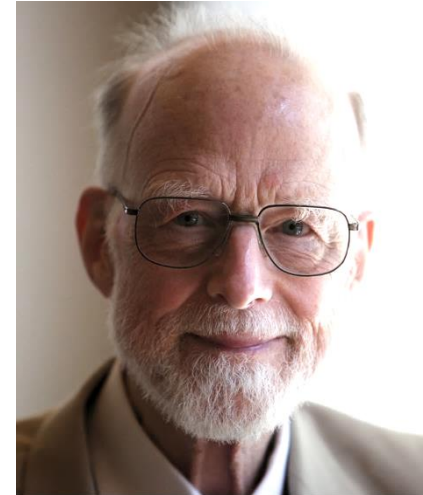
Betriebssysteme



Synchronisation als Sprachkonstrukt (C. A. R. Hoare, 1974)

Idee

- Korrekte Synchronisation erleichtern durch Erweiterung der Programmiersprache
- Monitor kapselt Daten und Operationen wie eine Klasse
- Zusätzliche Garantie: nur eine Operation gleichzeitig aktiv (wie Auto-Mutex)
- Zusätzlich `wait()` und `signal()` wie mit Auto-Condition Variable



Signalsemantik „Signal-and-Wait“ (Hoare)

- `signal()` weckt einen wartenden Thread und übergibt ihm **sofort** den Monitor
- Signalgeber wartet auf einer *Urgent Queue*, bis der Geweckte den Monitor verlässt
- Vorteil: Bedingung gilt garantiert direkt nach `wait()` – Prüfung mit `if` reicht

Alternative: „Signal-and-Continue“ (Mesa, Java, POSIX)

- Signalgeber läuft nach `signal()` weiter
- Geweckter muss erneut um den Monitor konkurrieren
- Bedingung kann nicht mehr erfüllt sein, wenn Geweckter weiterläuft, daher immer in `while`-Schleife prüfen

Java mit `synchronized` und `wait/notify`

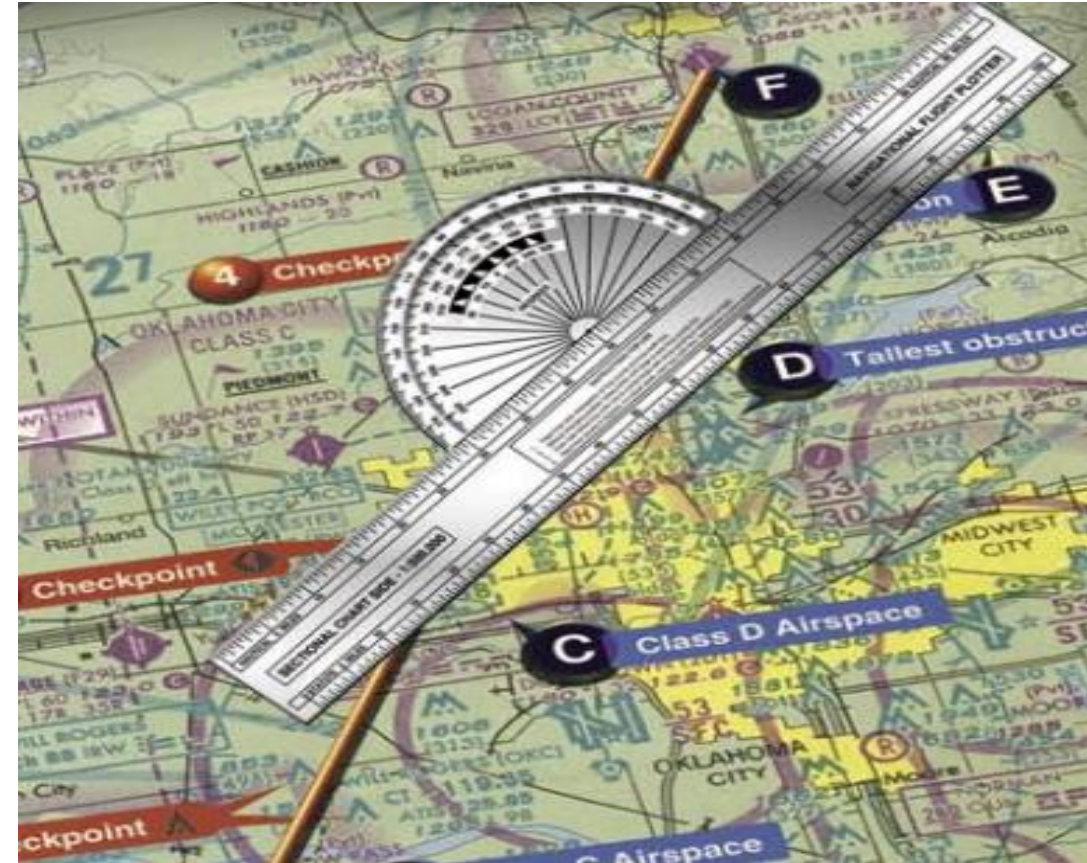
- Jedes Java-Objekt besitzt einen `Mutex` (und eine zugehörige `Wait-Set-Queue`)
- `synchronized` Methode/-Block sperrt den Monitor
- `wait()` gibt den Monitor wieder frei und blockiert
- `notify()/notifyAll()` weckt Wartende (Signal-and-Continue, Prüfung Wartebedingung in `while`)
- Nur **eine** implizite Condition pro Objekt; feingranularer: **ReentrantLock** mit mehreren **Condition**-Objekten

```
class BoundedBuffer { // ähnlich Producer-Consumer
    private Queue<Item> q = new ArrayDeque<>(); private static final int CAPACITY = 16;

    public synchronized void put(Item x) throws InterruptedException {
        while (q.size() == CAPACITY) wait(); // Monitor freigeben, warten
        q.add(x); notifyAll();               // Geweckte prüfen erneut (while!)
    }
    public synchronized Item take() throws InterruptedException {
        while (q.isEmpty()) wait();
        Item x = q.remove(); notifyAll();
        return x;
    }
}
```

Interprozesskommunikation

- Nebenläufigkeit und kritische Abschnitte
- Grundlegende IPC-Mechanismen
 - Semaphore und Mutex
 - Condition Variable
 - Monitor
- ➔ Weitere IPC-Mechanismen
 - Semaphoren, Shared Memory, Named und Unnamed Pipes/Fifos, Message Passing
- Klassische IPC-Probleme



Nur zu Lehrzwecken



Betriebssysteme



POSIX Named Semaphore für Prozesse

- Auch Prozesse können Semaphoren verwenden!
- Realisiert über Dateisystem:
 - Kernel-Objekt, erreichbar über Namen wie z.B. /mysem (im Filesystem in /dev/shm)
 - Mehrere Prozesse öffnen denselben Namen
- Operationen
 - sem_wait – P-Operation, blockierend
 - sem_post – V-Operation, weckt einen Wartenden
 - sem_trywait – nicht-blockierend
 - sem_timedwait – mit Timeout
- Weitere API-Varianten
 - Unnamed Semaphore mit sem_init in Shared Memory, für verwandte Prozesse
 - System V (semget/semop): älter, Semaphor-Mengen, Key statt Name

```
// POSIX Named Semaphores

#include <semaphore.h>
#include <fcntl.h>          // O_CREAT

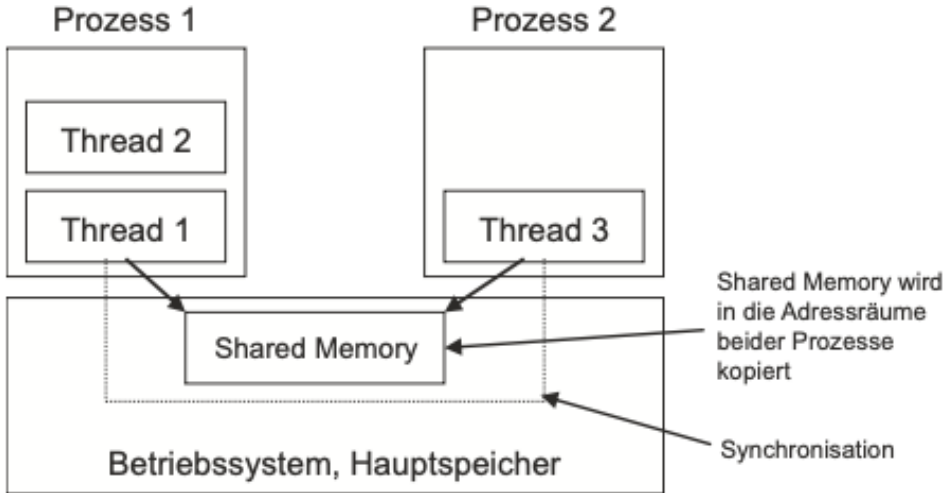
int main() {
    sem_t *s = sem_open("/mysem", O_CREAT, 0644, 1);
    if (s == SEM_FAILED)
        perror("open");

    sem_wait(s); // P: dekrementieren/blockieren
    // ... kritischer Abschnitt ...
    sem_post(s); // V: inkrementieren

    sem_close(s);          // Handle schließen
    sem_unlink("/mysem"); // Namen entfernen
}
```

POSIX Shared Memory (SHM)

Idee

- Gemeinsamer Speicherbereich, der von mehreren Prozessen genutzt wird
 - Schnellste Form der IPC
 - Kein Kopieren von Daten zwischen den Prozessen notwendig
- 
- Kernel-Objekt, erreichbar über Namen wie z.B. /myshm (im Filesystem in /dev/shm)
 - Erzeugen/Öffnen mit `shm_open()` ähnlich `open()`, liefert Filedeskriptor
 - Schließen mit `close()`
 - Löschen mit `shm_unlink()`
 - Prozess kann den geteilten Speicher an beliebige Stellen des eigenen virtuellen Adressraums einblenden mit `mmap()` und `munmap()`
 - Keine automatische Synchronisation, gleichzeitiger Zugriff führt zu Race Conditions
 - Synchronisation mit Semaphoren oder Funktionen für Datei-Locking `flock()`

Named und Unnamed Pipes

Idee

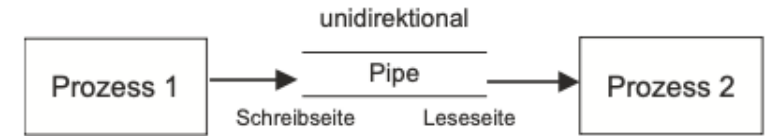
- Unidirektionaler (Halbduplex) Kommunikationskanal zwischen Prozessen
- Realisierung über Puffer im Hauptspeicher
- Neben Datenaustausch auch Synchronisation:
 - Schreiben/write() blockiert, wenn Puffer voll
 - Lesen/read() blockiert, wenn Puffer leer
 - read()-Prozess erhält **EOF**, wenn write()-Prozess die Pipe schließt
 - Signal **SIGPIPE** an write()-Prozess, wenn kein read()-Prozess

Unnamed Pipes (aus der Shell als Pipes | bekannt)

- Kommunikation nur zwischen Eltern- und Kindprozessen nach dem FIFO-Prinzip (heißt aber nicht so)
- Erzeugung mit `int pipe(int fd[2])`, dann Filedescriptor fd[0] zum Lesen und fd[1] zum Schreiben

Named Pipes (auch FIFOs genannt)

- Kommunikation zwischen beliebigen Prozessen, da Pipe-Name im Dateisystem angelegt (Dateityp p)
- Erzeugung in der Shell über mkfifo oder mit `int mkfifo(const char *pathname, mode_t mode);`



```
#include <unistd.h>

int fd[2];
pipe(fd); // Erstellt die Pipe

if (fork() == 0) { // Kindprozess: Schreiber
    close(fd[0]); // Leseende schließen
    write(fd[1], "Hallo Pipe!", 10);
    close(fd[1]);
} else { // Elternprozess: Leser
    close(fd[1]); // Schreibende schließen
    char buf[10];
    read(fd[0], buf, 5);
    close(fd[0]);
}
```

POSIX Message Queues

Idee

- Nachrichtenbasierte Kommunikation (statt byteweise) mit Prioritäten
- Nachrichten werden in Warteschlange im Kernel gespeichert
- Durch feste Nachrichtenstruktur typsicherer als Pipes
- Automatische Synchronisation durch den Kernel
- Im Dateisystem unter `/dev/mqueue` sichtbar

- Beispiel:

```
mqd_t mq = mq_open("/meine_queue", O_CREAT | O_WRONLY, 0666, NULL);
mq_send(mq, "Hallo", 5, 1 /* Priorität */);
mq_close(mq);
mq_unlink("/meine_queue");
```

- Häufig wichtig: Mehrere Leser und/oder Schreiber möglich
- Asynchrone (nicht-blockierende) Benachrichtigung möglich

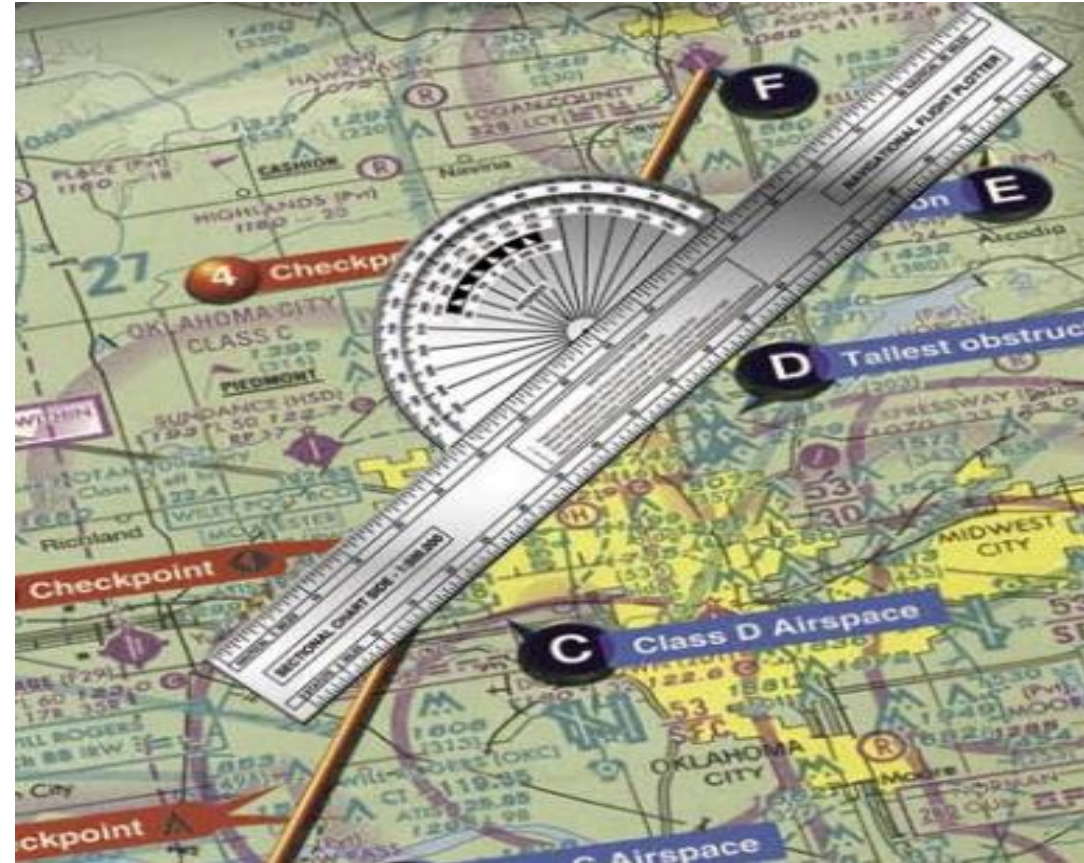
IPC in der Praxis

- Grundlegende IPC-Mechanismen insbesondere für Threads werden so genutzt
- Angegebene *weitere IPC-Mechanismen* häufig ersetzt/abstrahiert durch
 - TCP/UDP Sockets zur netzwerkbasierte Kommunikation (auch lokal)
 - Unix Domain Sockets (De-facto Standard: Docker, systemd, D-Bus, X11/Wayland, PostgreSQL)
 - D-Bus (im Linux Desktop und Systemdiensten)
- Nächste Stufe: Kommunikation über das Netzwerk
 - Netzwerkbasierte Message Broker (z.B. RabbitMQ, Kafka)

Interprozesskommunikation

- Nebenläufigkeit und kritische Abschnitte
- Grundlegende IPC-Mechanismen
 - Semaphore und Mutex
 - Condition Variable
 - Monitor
- Weitere IPC-Mechanismen
 - Semaphoren, Shared Memory, Named und Unnamed Pipes/Fifos, Message Passing

➔ Klassische IPC-Probleme



Nur zu Lehrzwecken



Betriebssysteme



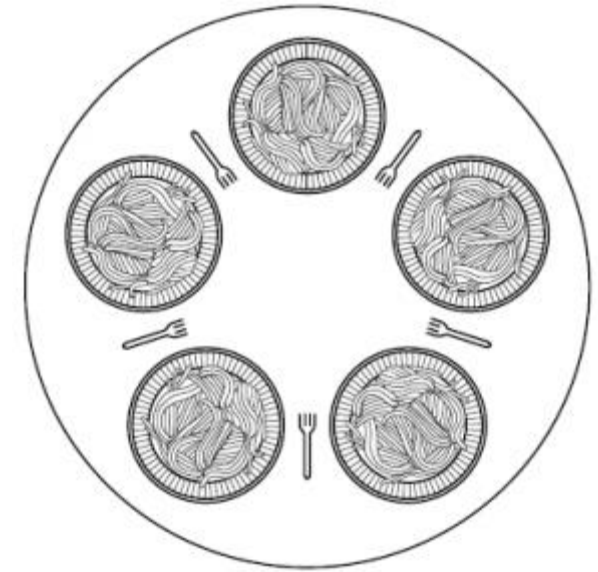
Producer/Consumer-Problem

Kennen wir schon – siehe Kapitel *Semaphore und Mutex* sowie *Condition Variable*:

- Zwei nebenläufige Prozesse (Producer und Consumer) nutzen zur Kommunikation einen gemeinsamen Puffer fester Größe
- Producer generiert Objekte und legt diese in einen Puffer
- Consumer entnimmt Objekte aus dem Puffer und verarbeitet diese
- Puffer hat eine begrenzte Größe:
 - Producer blockiert, wenn Puffer voll
 - Consumer blockiert, wenn Puffer leer
- Puffer ist eine gemeinsam genutzte Ressource
 - ➔ Zugriff muss synchronisiert werden

Problem der speisenden Philosophen

- $N=5$ Philosophen sitzen an einem runden Tisch mit N Gabeln und N Tellern Nudeln
- Philosophen denken oder essen, benötigen zum Essen jedoch zwei Gabeln
- Jeder Philosoph wird als Thread realisiert
- Vermeidung von Deadlocks – was passiert, wenn alle Philosophen gleichzeitig die linke Gabel aufnehmen?



```
const int N = 5;                                /* number of philosophers */
void philosopher(int i)                          /* i: philosopher number, from 0 to 4 */
{
    while (true) {
        think( );                               /* philosopher is thinking */
        take_fork( i );                         /* take left fork */
        take_fork( (i+1) % N );                 /* take right fork */
        eat( );                                 /* yum-yum, spaghetti */
        put_fork( i );                          /* put left fork back on the table */
        put_fork( (i+1) % N );                  /* put right fork back on the table */
    }
}
```

[Tanenbaum]

Reader/Writer-Problem

- Modelliert Zugriff auf Datenbanken
- Beliebige viele **Writer** greifen schreibend auf ein Objekt zu.
- Beliebige viele **Reader** greifen lesend auf das Objekt zu.
- Bei einem Schreibzugriff darf kein anderer Reader oder Writer aktiv sein (Gefahr, inkonsistente Daten zu lesen, bei Unterbrechung Writer mitten im Schreibvorgang).
- Wenn kein Writer aktiv ist, dürfen beliebige viele Reader gleichzeitig aktiv sein.
- Lösung mit
 - einer Variablen `num_active_readers` für die Anzahl der aktiven Leser
 - einer Semaphore `data_access` für den Zugriff auf die Daten zum Schreiben oder zum Lesen
 - einem Mutex `mutex` für den Zugriff auf `num_active_readers` in den Lesern – im kritischen Abschnitt die Semaphore `data_access` manipulieren
- siehe z.B. *Tanenbaum: Moderne Betriebssysteme*

Reader/Writer-Problem

- Modelliert Zugriff auf Datenbanken
- Beliebige viele **Writer** greifen schreibend auf ein Objekt zu.
- Beliebige viele **Reader** greifen lesend auf das Objekt zu.
- Bei einem Schreibzugriff darf kein anderer Reader oder Writer aktiv sein (Gefahr, inkonsistente Daten zu lesen, bei Unterbrechung Writer mitten im Schreibvorgang).
- Wenn kein Writer aktiv ist, dürfen beliebige viele Reader gleichzeitig aktiv sein.
- Lösung mit
 - einer Variablen `num_active_readers` für die Anzahl der aktiven Leser
 - einer Semaphore `data_access` für den Zugriff auf die Daten zum Schreiben oder zum Lesen
 - einem Mutex `mutex` für den Zugriff auf `num_active_readers` in den Lesern – im kritischen Abschnitt die Semaphore `data_access` manipulieren
- siehe z.B. *Tanenbaum: Moderne Betriebssysteme*

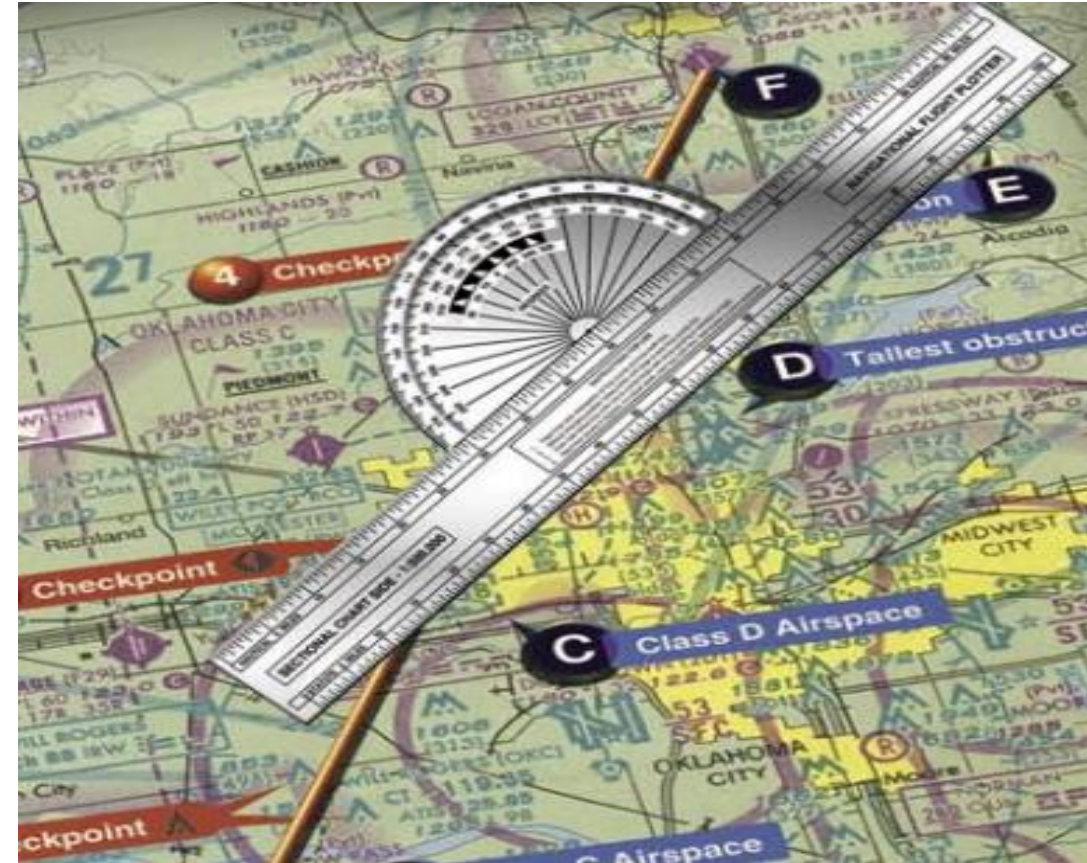
Der schlafende Friseur

- Friseursalon: Schneideraum mit einem Stuhl plus Warteraum mit N Stühlen
- Friseur bedient Kunden im Schneideraum und schaut dann im Warteraum nach weiteren Kunden
 - Weiterer Kunde wird in den Schneideraum gebracht und frisiert
 - Warteraum leer: Friseur legt sich im Schneideraum schlafen
- Neuer Kunde sieht nach, ob Friseur arbeitet oder schläft
 - Friseur arbeitet: Kunde setzt sich auf Stuhl im Warteraum bzw. geht, wenn alle Stühle belegt
 - Friseur schläft: Kunde weckt den Friseur und setzt sich in den Schneideraum
- Kritisch:
 - Zeitliche Überschneidung des Fertigfrisierens des letzten Kunden und des Ankommens eines neuen Kunden
 - Gleichzeitiges Ankommen mehrerer Kunden
- siehe z.B. *Tanenbaum: Moderne Betriebssysteme*



Interprozesskommunikation

- Nebenläufigkeit und kritische Abschnitte
- Grundlegende IPC-Mechanismen
 - Semaphore und Mutex
 - Condition Variable
 - Monitor
- Weitere IPC-Mechanismen
 - Semaphoren, Shared Memory, Named und Unnamed Pipes/Fifos, Message Passing
- Klassische IPC-Probleme



Concurrency ohne Threads – kooperatives Multitasking

Grundidee

- Ein einziger Thread verwaltet viele Aufgaben (Coroutines) – keine echte Parallelität
- Bei `await` gibt die Coroutine die Kontrolle freiwillig ab → Event-Loop führt andere Tasks aus
- Kein Race Conditions, da kritische Abschnitte nicht unterbrochen werden (enthalten kein `await`)

Event-Loop

- Zentrale Schleife (Library-Code): wartet auf I/O-Ereignisse und setzt blockierte Coroutines fort
- Python: `asyncio`-Modul; Node.js: `libuv`-Bibliothek (eingebaut)
- Parallele Ausführung mehrerer I/O-Operationen mit `asyncio.gather()` / `Promise.all()`

Vorteile gegenüber Threads / Prozessen

- Kein teurer Kontext-Switch des Betriebssystems, kein Stack-Overhead pro Thread
- Keine Mutexe / Semaphoren nötig – gemeinsamer Speicher wird nie gleichzeitig beschrieben
- Skaliert auf tausende gleichzeitige Verbindungen (z. B. Webserver)

Grenzen

- Ungeeignet für CPU-intensive Aufgaben → dort weiterhin Threads / Prozesse verwenden
- Blockierende Bibliotheksaufrufe (z. B. `synchrones open()`) frieren den gesamten Event-Loop ein

Kooperatives Multitasking im Single-Thread: I/O-Wartezeiten sinnvoll nutzen

Python (asyncio)

```
import asyncio
import aiohttp

async def fetch_url(session, url):
    async with session.get(url) as resp:
        return await resp.text()

async def main():
    urls = ["https://example.com", "https://python.org"]
    async with aiohttp.ClientSession() as session:
        tasks = [fetch_url(session, u) for u in urls]
        results = await asyncio.gather(*tasks) # parallel!

asyncio.run(main()) # Event-Loop starten
```

Node.js (Promise)

```
const fetch = require("node-fetch");

async function fetchUrl(url) {
    const resp = await fetch(url);
    return await resp.text();
}

async function main() {
    const urls = ["https://example.com",
                  "https://nodejs.org"];
    const tasks = urls.map(u => fetchUrl(u));
    const results = await Promise.all(tasks); //parallel
}

main(); // Event-Loop starten
```

- Keine Parallelität, keine präemptive Nebenläufigkeit: ein Thread, kooperatives Scheduling via Event-Loop
- Ideal für I/O-bound Tasks (HTTP, DB, Dateisystem)
- Ungeeignet für CPU-bound Tasks, nutzt keine Multicore-CPU's aus

Leichtgewichtige Threads der Go-Laufzeitumgebung mit Channel-Kommunikation

Grundidee

- `go func()` startet eine neue Goroutine (-> Go-Runtime)
- ~2 KB Stack-Start, wächst dynamisch
- M:N-Scheduling auf OS-Threads
- Echte Parallelität auf mehreren CPU-Kernen (GOMAXPROCS)

Kommunikation via Channels

- `chan`-Typen: typsicherer Nachrichtenkanal zwischen Goroutines
- Blockierendes Senden/Empfangen parkt die Goroutine
- `select` für nicht-blockierendes Multiplexing mehrerer Channels

Vorteile / Grenzen

- Einfache Syntax, kein `async`-Refactoring
- Race-Detector eingebaut
- Channels machen Shared Memory unnötig – bei Bedarf aber `sync.Mutex` verfügbar

Go – Goroutines & Channels

```
package main
import ("fmt"; "sync")

func worker(id int, wg *sync.WaitGroup) {
    defer wg.Done() // am Ende der F'on ausführen
    fmt.Printf("worker %d\n", id) // I/O-Task
}

func main() {
    var wg sync.WaitGroup
    for i := 0; i < 5; i++ {
        wg.Add(1)
        go worker(i, &wg) // Goroutine starten!
    }
    wg.Wait() // auf alle warten
}

// Channel-Beispiel:
ch := make(chan int, 1) // gepufferter Channel
go func() { ch <- 42 }() // senden
val := <-ch // empfangen (blockiert)
```

Project Loom (Java 21+) – Leichtgewichtige Threads im JDK

Grundidee

- Virtual Threads sind `java.lang.Thread`-Instanzen, nicht 1:1 an OS-Threads gebunden
- JVM multiplext viele Virtual Threads auf wenige **Platform Threads** (OS-Threads)
- Bei blockierendem I/O: JVM parkt Virtual Thread – Platform Thread läuft weiter

Vergleich mit klassischen Threads

- Platform Thread: ~1 MB Stack, teurer OS-Kontext-Switch, max. ~10.000 Threads (Grenze Betriebssystem)
- Virtual Thread: kleiner Heap-Stack, kein OS-Overhead, Millionen möglich
- Teil-Nutzung von OS-Threads → echter Parallelismus auf mehreren CPU-Kernen – im Ggs. zu `async/await`

Vorteile

- Sync. Code läuft unverändert – kein `async/await`-Refactoring nötig
- Hoher Durchsatz bei I/O-bound Workloads, einfache `try/catch`-Fehlerbehandlung

Grenzen

- CPU-intensive Tasks profitieren nicht (kein Parken möglich)
- Pinning: `synchronized`-Blöcke können Virtual Thread am OS-Thread festhalten

JVM multiplext Virtual Threads auf Platform Threads – Java 21 (LTS)

ExecutorService (Java 21)

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.net.http.*; // Java 11+ HTTP-Client

public class VirtualThreadDemo {
    public static void main(String[] args) {

        // Executor erstellt Virtual Threads
        try (ExecutorService ex =
            Executors.newVirtualThreadPerTaskExecutor()) {

            for (int i = 0; i < 1000; i++) {
                int id = i;
                ex.submit(() -> { // Lambda = Task
                    fetchAndPrint(id); // blockiert → JVM parkt VT
                });
            }
        } // try-with-resources wartet auf alle Tasks
    }
}
```

Thread.ofVirtual() API

```
static void fetchAndPrint(int id) {
    var client = HttpClient.newHttpClient();
    var req = HttpRequest.newBuilder()
        .uri(URI.create("https://example.com/" + id))
        .build();
    var res = client.send(req,
        HttpResponse.BodyHandlers.ofString());
    System.out.println("id=" + id + " " +
        res.statusCode());
}

// Alternativ: einzelner Virtual Thread
Thread vt = Thread.ofVirtual()
    .name("worker-", 0)
    .start(() -> fetchAndPrint(42));
vt.join(); // warten bis fertig
```

- Echte Parallelität auf mehreren CPU-Kernen
- JVM parkt blockierte VTs automatisch
- Kein Refactoring bestehender sync. Bibliotheken nötig.

Falsche Realisierung kritischer Abschnitte ohne interested

```
const int N = 2;           // Zwei Prozesse
int turn;                  // Prozess 0 oder 1, wer ist am Zug?

void enter_region (int process) // process is 0 or 1
{
    int other = 1 - process;    // Nummer des anderen Prozesses
    turn = other;              // Flag setzen
    while (turn == other)
    {
        // do nothing (spinlock)
    }
}

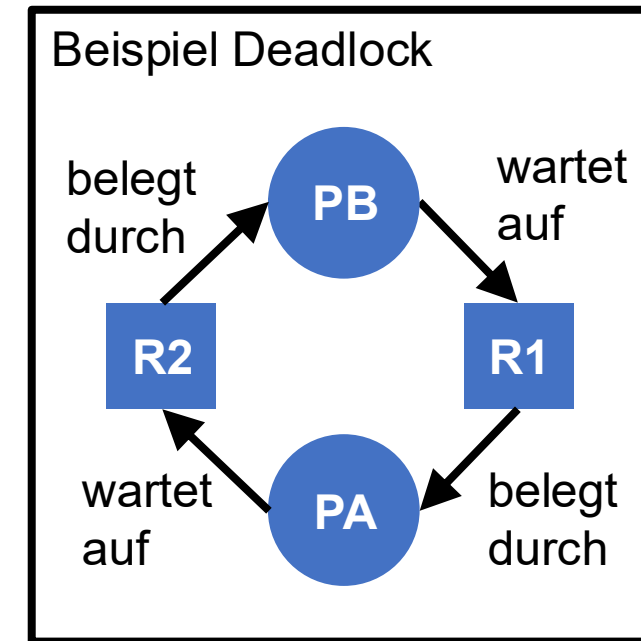
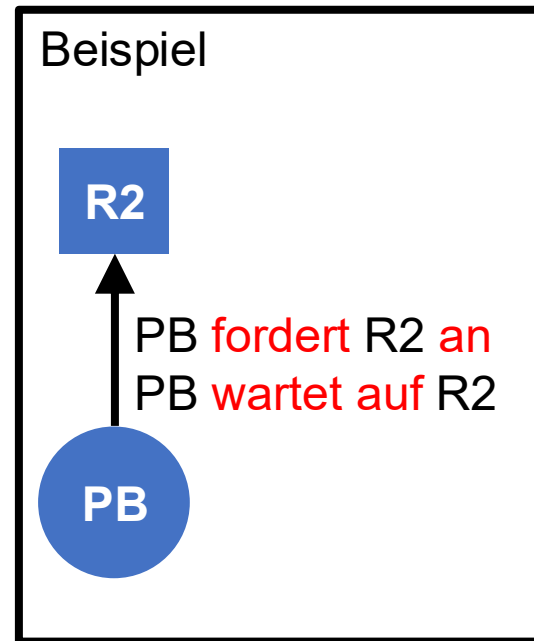
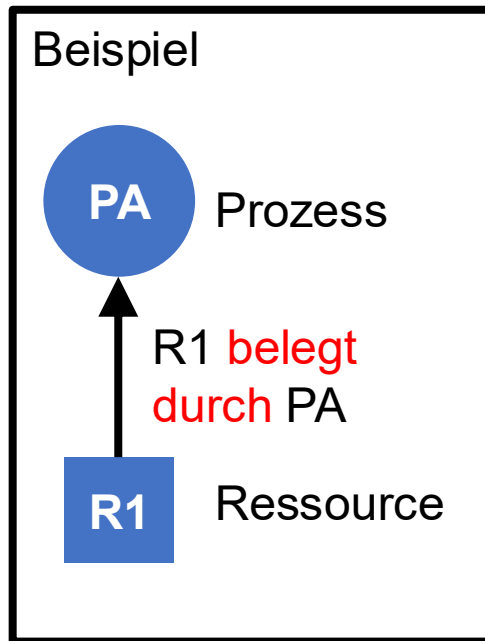
void leave_region(int process)
{
    turn = 1 - process;        // Flag setzen
}
```

- wartet immer, bis der andere Prozess interessiert ist (Dekker) – im Zweifel also ewig!
- Auch mit `turn=process` (statt `turn=other`) fehlerhaft, dann können beide Prozesse nacheinander die Critical Section betreten!
- Durch Sperren von Interrupts lässt sich ein Taskwechsel unterbinden; korrekte Implementierung von `enter/leave` dann einfach möglich (aber: Interrupt-Latenz)

[Tanenbaum]



Deadlocks erkennen mit Belegungs-Anforderungs-Graph



- **Zyklen** im Belegungs-Anforderungs-Graphen bedeuten **Deadlocks**
- Zyklen **lassen sich durch das Betriebssystem zur Laufzeit erkennen** (Prüfung des Deadlock-Kriteriums *zyklische Wartebedingung*)
- System könnte dann Reset auslösen oder die beteiligten Prozesse und Ressourcen terminieren und neu starten

Realisierung kritischer Abschnitte: Peterson (1981)

```
const int N = 2;           // Zwei Prozesse
int turn;                  // Prozess 0 oder 1, wer ist am Zug?
bool interested[N] = {false};

void enter_region (int process) // process is 0 or 1
{
    int other = 1 - process;    // Nummer des anderen Prozesses
    interested[process] = true; // Interesse zeigen
    turn = other;               // Flag setzen
    while (interested[other] == true && turn == other)
    {
        // do nothing (spinlock)
    }
}

void leave_region(int process)
{
    interested[process] = false;
}
```

- Bitte nicht selbst so implementieren, Nachteile:
 - Aktives Warten verschwendet Rechenzeit (Sperrung mit akt. Warten =: Spinlock)
 - Prioritätsgesteuertes Scheduling bei hochpriorem wartenden Prozess → Deadlock
- Bessere Lösungen durch spezielle Funktionen des Betriebssystems:
 - Mutex (MUTual Exclusion) für gegenseitigen Ausschluss (Critical Sections)
 - Semaphoren für Signalisierung

[Tanenbaum]